Measuring the Indic Tokenizer Tax: A Comparative Study of Sarvam AI's Tokenizers and Akshara-Aware Subword Segmentation for Indian Languages

Nikhileshwar Jakka
Department of Computer Science and Engineering
Manipal University Jaipur, Jaipur, India
jkk.nikhil@gmail.com

***Abstract*—The same sentence can cost very different numbers of tokens in different languages, which changes API cost, usable context and latency. We measure this "tokenizer tax" for 11 Indian languages on the parallel FLORES-200 devtest set across 12 published tokenizers, focusing on the three generations of Sarvam AI models. Using exact UTF-8 byte spans for every token, we also measure how often tokenizers break *aksharas*, the orthographic syllables of Brahmic scripts. We find that Sarvam-M reuses the Mistral-Small-3.1 vocabulary unchanged (all 131,072 entries), and that the Sarvam-30B/105B vocabulary shares 91.3% of its entries, with identical ids, with Gemma-3. Compared with Sarvam-1, Sarvam-30B lowers fertility on 5 Indo-Aryan languages but raises it on all four Dravidian languages, widening the Dravidian/Indo-Aryan fertility ratio from 1.05 to 1.32. Byte-level tokenizers such as Llama-3.1 split 91.4% of aksharas. Finally, we propose *akshara-atomic BPE*, which maps each frequent akshara to a single base symbol. At 64K vocabulary it reduces the akshara split rate from 8.3% to 0.6% (a 92.9% reduction) at the price of 5.1% higher fertility, and every remaining split falls on an akshara outside the mapped inventory.**

***Index Terms*—tokenization, Indian languages, BPE, fertility, akshara, Sarvam AI, multilingual fairness**

I. Introduction

Large language models (LLMs) see text only through their tokenizer. When a tokenizer needs more tokens for one language than for another, speakers of that language pay more per request, fit less text into the context window and wait longer for output [1], [2]. On the parallel FLORES-200 devtest set, GPT-4o's `o200k` tokenizer needs 1.93× as many tokens for Telugu as for the same content in English, and Llama-3.1 needs 8.29×. Indian scripts add a second, less visible cost. Brahmic scripts are written in *aksharas* (orthographic syllables such as ಕ್ಷ or ശ്ല) that combine consonants, viramas and vowel signs. A tokenizer can place a boundary *inside* an akshara, leaving a vowel sign or virama without its consonant.

Sarvam AI has released three generations of models with three different tokenizers: Sarvam-1 (Oct. 2024) with a custom 68K Indic vocabulary [3], Sarvam-M (May 2025), built on Mistral Small [4], and the open-weight Sarvam-30B and 105B (Mar. 2026) [5]. This lineage is a rare natural experiment in how an Indian lab's tokenizer design changed over time. Our contributions are:

- A reproducible benchmark of 12 tokenizers on 11 Indian languages, with exact byte-span adapters for four tokenizer families, bootstrap confidence intervals and paired significance tests.
- An analysis of the Sarvam lineage that traces the Sarvam-M and Sarvam-30B/105B vocabularies to their sources (Mistral-Small-3.1 and Gemma-3) and quantifies the effect on Dravidian languages.
- The *Akshara Split Rate* (ASR), a script-aware intrinsic metric, measured for every tokenizer.
- *Akshara-atomic BPE*, which removes akshara splits by construction for its mapped inventory, with a study of how fertility trades off against vocabulary size.

Code, data pointers and all results are public.[1]

II. Background and Related Work

Subword tokenization. Byte-pair encoding (BPE) merges frequent symbol pairs [6]; the Unigram LM tokenizer prunes a large vocabulary probabilistically [7]; SentencePiece implements both directly on raw text [8]. Byte-level BPE, as in tiktoken [9] and Llama 3 [10], operates on UTF-8 bytes and can therefore split a single character.

Measuring tokenizers. Rust et al. introduced fertility and the proportion of continued words and linked them to downstream quality [11]. Petrov et al. defined tokenization parity on parallel text [1], and Ahia et al. showed the resulting cost disparity in commercial APIs [2]. Zouhar et al. proposed Rényi efficiency [12].

Indian-language resources. IndicNLPSuite [13], IndicBERTv2 and IndicXTREME [14], MuRIL [15], IndicTrans2 with the IN22 benchmark [16], and the Sangraha corpus of IndicLLMSuite [17] underpin most Indic NLP. FLORES-200 [18], [19] provides professionally translated parallel sentences.

[1] https://github.com/jkknikhil-git/indic-tokenizer-study

TABLE I
TOKENIZERS EVALUATED. "SPANS" IS HOW EXACT TOKEN BYTE SPANS ARE RECOVERED.

Tokenizer	Algorithm	Vocab	Spans
Sarvam-1	SentencePiece BPE+byte-fallback	68,096	bytes
Sarvam-M	byte-level BPE	131,072	bytes
Sarvam-30B/105B	SentencePiece BPE+byte-fallback	262,144	bytes
GPT-4o (o200k)	tiktoken byte-level BPE	200,019	bytes
GPT-4 (cl100k)	tiktoken byte-level BPE	100,277	bytes
Llama-3.1	byte-level BPE	128,256	bytes
Gemma-3	SentencePiece BPE+byte-fallback	262,145	bytes
Qwen2.5	byte-level BPE	151,665	bytes
MuRIL	WordPiece	197,258	offsets
IndicBERTv2	WordPiece	250,000	offsets
XLM-R	SentencePiece Unigram	250,002	offsets
mBERT	WordPiece	119,547	offsets
Ours BPE 64K	SentencePiece bpe	64,000	bytes
Ours Akshara-BPE 64K	SentencePiece bpe (akshara-atomic)	64,000	bytes
Ours Unigram 64K	SentencePiece unigram	64,000	bytes

Indic tokenizers. Brahma et al. evaluated BPE and Unigram tokenizers on 17 Indian languages [20]. MorphTok adds morphology-aware pre-tokenization and a constrained BPE that stops dependent vowel signs from becoming standalone tokens [21]. Rana et al. build a multilingual tokenizer with an Indic variant [22]. Our akshara-atomic BPE is closest to MorphTok's constrained BPE, but it enforces whole-akshara units through the base alphabet rather than through merge constraints, and it applies to all nine Brahmic scripts we study. Our segmenter starts from Unicode extended grapheme clusters [23].

III. METHODOLOGY

A. Data

For evaluation we use FLORES-200 devtest (1,012 parallel sentences per language) for English and 11 Indian languages. The seven Indo-Aryan (IA) languages are Hindi, Marathi, Bengali, Assamese, Gujarati, Punjabi and Odia; the four Dravidian (Dr) languages are Telugu, Tamil, Kannada and Malayalam. IN22-Gen is gated on the Hugging Face Hub, so we left it out. To train our own tokenizers we streamed 50,000 sentences per language (600,000 in total, plus 1,200 held out) from the verified split of Sangraha [17]. We deduplicated them and removed any sentence that shares a normalised 40-character substring with FLORES dev or devtest (2 were removed).

B. Tokenizer zoo

Table I lists the tokenizers. We download tokenizer files only, never model weights. Llama-3.1 and Gemma-3 come from public mirrors, which we verified against the originals. The Llama-3.1 mirror's 128,000 token-to-id pairs match Meta's `llama-models` file exactly, and the Gemma-3 mirror's tokenizer files have the same git blob hashes as Google's repository. The Sarvam-30B and 105B vocabularies and merges are identical, so we report them as one row.

C. Byte-span adapters

Measuring akshara splits requires the exact UTF-8 byte span of every token. For tiktoken and byte-level BPE we map each token back to its raw bytes. For SentencePiece with byte fallback we convert the word-boundary meta-symbol (U+2581) back to a space and `<0xNN>` pieces back to bytes. In both cases we assert that the concatenated spans reproduce the tokenizer's normalised input. For WordPiece and Unigram without byte fallback we use the tokenizer's character offsets, after checking that they are monotonic and cover every non-whitespace character. Formatting characters (ZWJ/ZWNJ) that the normaliser deletes are allowed to stay uncovered. Validation succeeded for at least 12,119 of 12,144 sentences per tokenizer; the few failures are excluded from the span-based metrics only.

D. Metrics

For tokenizer T, language ℓ and sentence set S, with words obtained by whitespace splitting and punctuation stripping (the same rule for every tokenizer):

$$\text{Fertility}(T,\ell) = \textstyle\sum_{s\in S} |T(s_\ell)| \,/\, \sum_{s\in S} |W(s_\ell)| \tag{1}$$

$$\text{Parity}(T,\ell) = \textstyle\sum_{s\in S} |T(s_\ell)| \,/\, \sum_{s\in S} |T(s_{\text{en}})| \tag{2}$$

$$\text{ASR}(T,\ell) = 100 \cdot \frac{|\{a : \exists b \in B_T,\ a_{\text{start}} < b < a_{\text{end}}\}|}{|\{a\}|} \tag{3}$$

Here a ranges over aksharas and B_T is the set of token boundaries in bytes. We also report the proportion of continued words (PCW) [11], the intra-character split rate (the share of codepoints whose bytes are split), the number of words that fit in an 8,192-token context, the share of `[UNK]` tokens, and Rényi efficiency (α=2.5) [12]. We compute 95% confidence intervals with 1,000 bootstrap resamples of sentences. We compare tokenizers with paired Wilcoxon signed-rank tests over FLORES sentences, Holm-corrected, and report the rank-biserial correlation r as an effect size.

E. Akshara segmentation

We first split text into extended grapheme clusters [23], then split those clusters again at every virama→consonant point, so the result does not depend on the Unicode version. We then join a cluster to the next one when it ends in a virama (canonical combining class 9), optionally followed by ZWJ/ZWNJ, and the next cluster starts with a consonant of the same script. This turns ಕ್ಷ, ದ್ರ and ಕ್ಕ into single aksharas. Two rules are script-specific. In Tamil the pulli marks a visible dead consonant, so க்+க stays two units; the exception is க்ஷ. In Malayalam, chillus (atomic or virama+ZWJ) are standalone. The segmenter passes 30 hand-checked examples covering all nine scripts and Latin.

F. Akshara-atomic BPE

We count every Indic akshara in the training corpus. Of these, 22,221 types occur at least twice. We map the most frequent $\min(n_{\geq 2}, V/2)$ types to codepoints in Supplementary Private Use Area-A (U+F0000 onwards); the cap keeps the base alphabet from crowding out merges at small vocabulary sizes V. SentencePiece BPE is then trained on the mapped text, and `encode`/`decode` apply and reverse the

TABLE II
MEAN FERTILITY, PARITY AND ASR OVER INDO-ARYAN (IA) AND DRAVIDIAN (DR) LANGUAGES; EN = ENGLISH FERTILITY.

	Fertility			Parity		ASR (%)	
Tokenizer	En	IA	Dr	IA	Dr	IA	Dr
Sarvam-1	1.44	2.27	2.39	1.50	1.22	10.2	9.6
Sarvam-M	1.27	5.43	3.99	3.98	2.30	46.5	38.9
Sarvam-30B/105B	1.24	1.93	2.56	1.49	1.52	7.9	11.0
GPT-4o (o200k)	1.23	3.04	3.27	2.35	1.96	29.7	29.4
GPT-4 (cl100k)	1.24	9.23	14.26	7.10	8.43	94.3	100.0
Llama-3.1	1.24	8.36	14.26	6.41	8.43	86.4	100.0
Gemma-3	1.24	2.58	2.96	2.00	1.76	16.5	14.1
Qwen2.5	1.26	8.11	11.70	6.17	6.81	84.2	83.7
MuRIL	1.26	1.53	1.93	1.18	1.13	4.7	5.7
IndicBERTv2	1.24	1.46	1.81	1.14	1.08	3.4	5.2
XLM-R	1.40	2.12	2.57	1.45	1.35	8.2	9.1
mBERT	1.32	2.65	4.07	1.93	2.26	20.6	25.5
Ours BPE 64K	1.60	1.78	2.32	1.07	1.07	7.0	10.5
Ours Akshara-BPE 64K	1.69	1.86	2.46	1.06	1.07	0.6	0.5
Ours Unigram 64K	1.58	1.76	2.28	1.07	1.06	5.5	7.9

mapping. Merges can join aksharas into larger tokens but can never split a mapped akshara. As baselines we train standard BPE with identical settings (identity normalisation, `character_coverage`=1.0, byte fallback) at 16K, 32K and 64K, and a 64K Unigram model. On 1,000 held-out sentences, `decode(encode(x))=x` holds for every model.

IV. RESULTS

Table II summarises the main metrics, and Fig. 1 gives fertility for every language.

A. RQ1: The Sarvam lineage

Sarvam-1 gives Indian scripts 73.0% of its vocabulary (49,707 entries). Its mean Indic fertility of 2.31 falls within the blog's claimed 1.4–2.1 range [3] for Hindi (1.40) and Punjabi (1.68). It is higher for Malayalam (2.86) and for Assamese (4.78), a language Sarvam-1 does not list as supported. **Sarvam-M** uses the Mistral-Small-3.1 (Tekken [24]) vocabulary unchanged: all 131,072 entries match in string and id, consistent with the failed vocabulary-extension experiments reported in its blog [4]. Its mean Indic fertility rises to 4.90, reaching 18.36 on Odia. **Sarvam-30B/105B** shares 239,314 entries (91.3%) with Gemma-3 [25], with identical ids. The remaining 22,828 slots replace mostly CJK (9,546), Latin (3,890) and Thai entries. They add Gurmukhi (3,527), Odia (3,302), Kannada (2,946), Gujarati (2,643), Telugu (2,530) and Malayalam (2,071) tokens, plus Meetei Mayek, Tibetan and Ol Chiki. No Devanagari, Bengali or Tamil tokens were added, so Sarvam-30B and Gemma-3 tokenize Hindi, Marathi, Bengali, Assamese and Tamil identically (e.g. Hindi fertility 1.39 for both). Overall, Indian scripts make up only 19.6% of the 262K vocabulary (51,355 entries).

Fig. 2 and Table III compare the generations. Sarvam-30B lowers fertility relative to Sarvam-1 on 5 languages (Hindi, Bengali, Assamese, Punjabi, Odia) but raises it on 6, including all four Dravidian ones; 11 of 11 differences are significant after Holm correction. Against general-purpose tokenizers, both Sarvam vocabularies beat GPT-4o (mean Indic fertility 3.12) and are far ahead of Llama-3.1 (10.50). The encoder vocabularies MuRIL and IndicBERTv2, which devote 71.8% and 69.1% of their entries to Indian scripts, remain the most compact.

TABLE III
SARVAM-1 VS. SARVAM-30B FERTILITY. PAIRED WILCOXON OVER 1,012 SENTENCES, HOLM-CORRECTED; $r < 0$ MEANS SARVAM-30B IS HIGHER.

Language	S-1	S-30B	p_{Holm}	r
Hindi	1.40	1.39	<0.001	0.16
Marathi	1.78	2.00	<0.001	-0.96
Bengali	2.07	1.69	<0.001	0.94
Assamese	4.78	2.83	<0.001	1.00
Gujarati	1.81	1.95	<0.001	-0.79
Punjabi	1.68	1.63	<0.001	0.23
Odia	2.36	1.99	<0.001	0.99
Telugu	2.14	2.33	<0.001	-0.71
Tamil	2.18	2.38	<0.001	-0.88
Kannada	2.38	2.55	<0.001	-0.69
Malayalam	2.86	2.97	<0.001	-0.42

B. RQ2: Indo-Aryan vs. Dravidian, and scripts

Dravidian fertility is significantly higher than Indo-Aryan fertility for 18 of 19 tokenizers (ratio of sentence-level means; Sarvam-1 1.05, Sarvam-30B 1.32, our BPE-64K 1.30). Part of this gap is morphological rather than a tokenizer artefact: agglutinative Dravidian words are longer, so a sentence has fewer of them. Parity controls for content and tells a different story. Dravidian parity exceeds Indo-Aryan parity in only 10 of 19 tokenizers. The largest gaps are in byte-level tokenizers with almost no Indic merges (Llama-3.1 ratio 1.32, GPT-4 1.19), where cost tracks the number of UTF-8 bytes. Script coverage matters more than language family. Odia is the most expensive language for GPT-4o (parity 4.98) and Sarvam-M (12.87), and Assamese is the most expensive for Sarvam-1 (2.92). Sarvam-30B is the only Sarvam tokenizer with higher Dravidian than Indo-Aryan parity (1.50 vs. 1.53, r=0.23).

C. RQ3: Broken aksharas

Fig. 3 shows the ASR. Byte-level tokenizers without Indic merges split nearly every akshara: GPT-4 96.4%, Llama-3.1 91.4% and Qwen2.5 84.0%. Llama-3.1 also splits 68.9% of codepoints into byte fragments. GPT-4o splits 29.6%, Sarvam-1 10.0% and Sarvam-30B 9.0%, while the WordPiece encoders MuRIL (5.1%) and IndicBERTv2 (4.1%) are lowest among published tokenizers. Splits fall disproportionately on Dravidian text in Sarvam-30B: its Dravidian ASR is 1.40× its Indo-Aryan ASR, and on Telugu it splits 11.5% of aksharas against 8.7% for Sarvam-1. Fig. 4 shows typical failures: vowel signs and viramas left as separate tokens, and hex byte fragments in Llama-3.1.

D. RQ4: Akshara-atomic BPE

Table IV and Fig. 5 compare our tokenizers. At every vocabulary size, akshara-atomic BPE removes most akshara

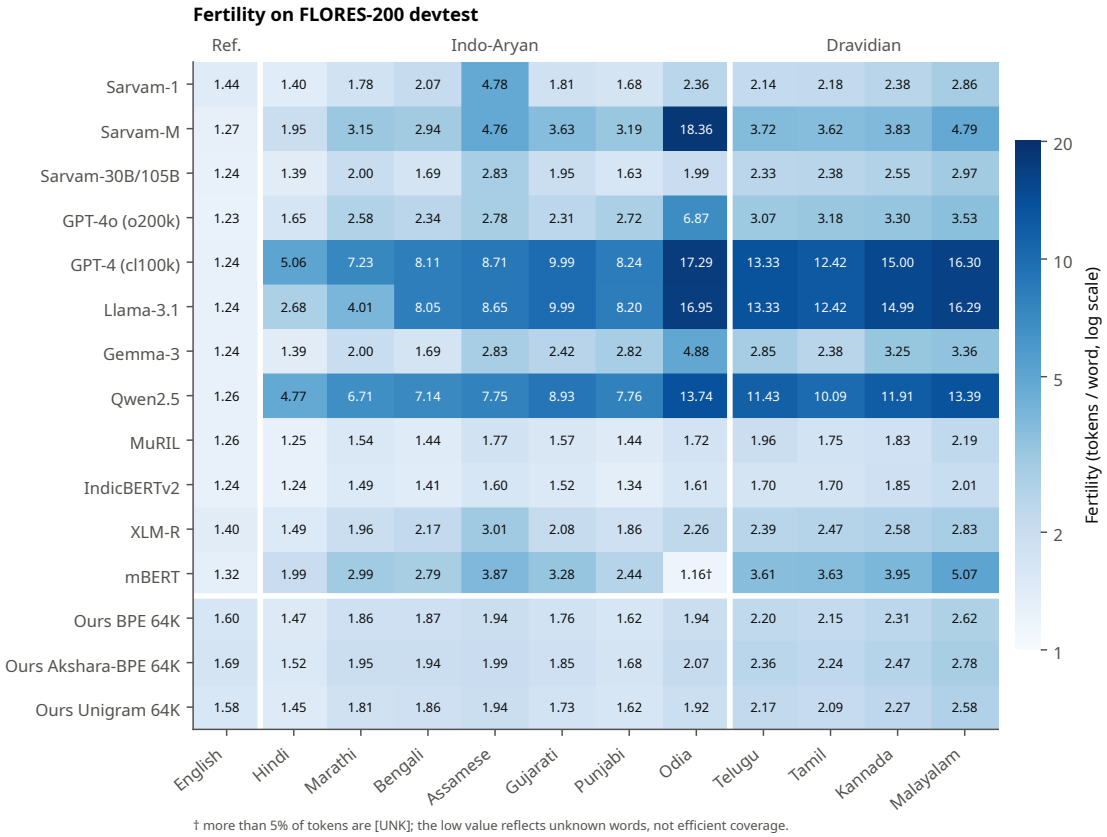


Fig. 1. Fertility (tokens per word) on FLORES-200 devtest. †: more than 5% `[UNK]` tokens (mBERT on Odia, 85.8%), so the low value is an artefact.

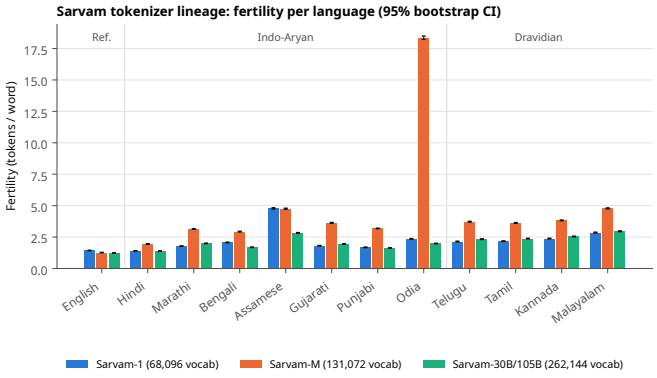


Fig. 2. Fertility of the three Sarvam tokenizers with 95% bootstrap CIs.

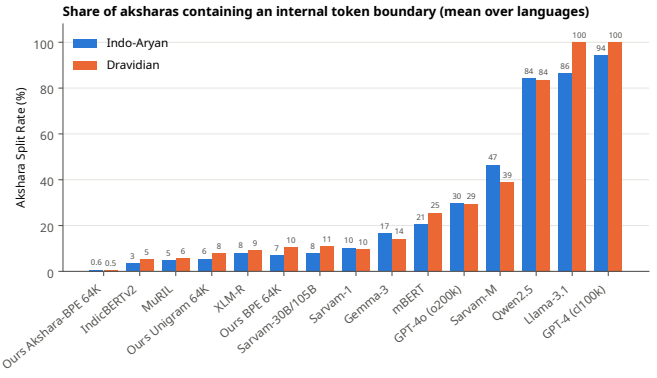


Fig. 3. Akshara Split Rate averaged over Indo-Aryan and Dravidian languages.

splits: from 15.5% to 1.6% at 16K and from 8.3% to 0.6% at 64K. Fertility rises by 7.2% and 5.1% respectively, and the cost shrinks as the vocabulary grows. Every remaining split falls on an akshara outside the mapped inventory: at 64K, 0 mapped aksharas are split, and 0.59% of FLORES aksharas are unmapped. The Dravidian/Indo-Aryan ASR ratio falls from 1.49 for standard BPE to 0.94. At a vocabulary close to Sarvam-1's, the akshara-atomic model has lower mean Indic fertility (2.08 vs. 2.31) and a 17× lower ASR. Training took 19.8 s for the 64K akshara-atomic model on a laptop CPU (plus mapping time), against 20.8 s for standard BPE. A fair comparison needs one caveat: our models were trained only on these 12 languages, and their English fertility (1.60) is worse than that of the commercial tokenizers, which flatters their parity.

V. Discussion

Practical implications. For Indian deployments, the tokenizer can matter as much as the model. At 8K tokens,

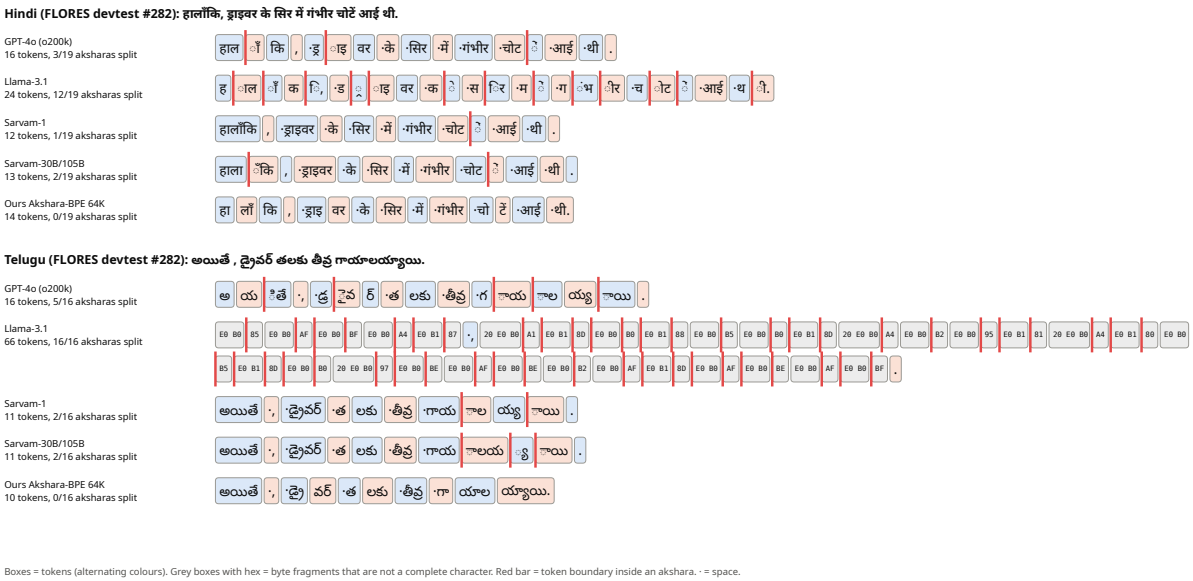


Fig. 4. Tokenization of one parallel FLORES sentence in Hindi and Telugu. Red bars mark token boundaries inside an akshara.

TABLE IV
OUR TOKENIZERS: MEAN OVER 11 INDIAN LANGUAGES. MAPPED = AKSHARA TYPES GIVEN THEIR OWN BASE SYMBOL.

Model	Fert.	Parity	ASR	PCW	Mapped
BPE 16K	2.64	1.08	15.5	0.65	–
Akshara-BPE 16K	2.84	1.06	1.6	0.71	8,000
BPE 32K	2.28	1.07	11.5	0.56	–
Akshara-BPE 32K	2.47	1.06	0.7	0.62	16,000
BPE 64K	1.98	1.07	8.3	0.47	–
Akshara-BPE 64K	2.08	1.07	0.6	0.50	22,221
Unigram 64K	1.95	1.07	6.4	0.42	–
Sarvam-1	2.31	1.40	10.0	0.48	–
Sarvam-30B/105B	2.16	1.50	9.0	0.50	–

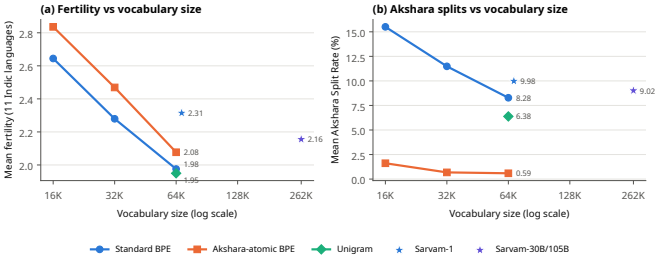


Fig. 5. Fertility and ASR vs. vocabulary size, with Sarvam-1 and Sarvam-30B as reference points.

Llama-3.1 fits about 615 Telugu words, against 3,821 for Sarvam-1 and 2,670 for GPT-4o. Because API pricing is per token, the parity values in Table II act directly as cost multipliers. Reusing a strong general vocabulary (Mistral for Sarvam-M, Gemma-3 for Sarvam-30B) brings English and code efficiency, but inherits that vocabulary's gaps unless the replaced slots cover every target script; Tamil, for example, received no new tokens. Akshara-atomic tokenization offers linguistically well-formed units at a modest fertility cost and could be combined with vocabulary transplantation.

Limitations. Words are defined by whitespace, which penalises agglutinative languages in fertility; we therefore also report parity. FLORES is a Wikipedia-style domain. All metrics are intrinsic: we did not train language models, so we cannot claim that fewer akshara splits improve downstream quality. Our akshara rules approximate orthographic syllables (e.g. Tamil pulli handling, nukta forms) and are not a full linguistic syllabifier. Our tokenizers were trained on 50K sentences per language, far less than production tokenizers.

VI. Conclusion and Future Work

Sarvam's tokenizers moved from a dedicated Indic vocabulary (Sarvam-1), to an unchanged general vocabulary (Sarvam-M), to a Gemma-3 vocabulary with 8.7% of its slots reassigned (Sarvam-30B/105B). The last step improved several Indo-Aryan and low-resource scripts but made all four Dravidian languages more expensive and more often split than with Sarvam-1. Akshara-atomic BPE suppresses akshara splits at a 5.1–7.2% fertility cost. Future work will train small language models on each tokenizer to test downstream effects, extend the study to IN22 and conversational domains, and combine akshara-atomic units with morphology-aware segmentation [21].

References

[1] A. Petrov, E. La Malfa, P. H. S. Torr, and A. Bibi, "Language model tokenizers introduce unfairness between languages," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 36 963–36 990. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/74bb24dca8334adce292883b4b651eda-Abstract.html

[2] O. Ahia, S. Kumar, H. Gonen, J. Kasai, D. Mortensen, N. Smith, and Y. Tsvetkov, "Do all languages cost the same? tokenization in the era of commercial language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 9904–9923. [Online]. Available: https://aclanthology.org/2023.emnlp-main.614/

[3] Sarvam AI, "Sarvam 1: the first Indian language LLM," https://www.sarvam.ai/blogs/sarvam-1, 24 October 2024, 2024.

[4] ——, "Sarvam-M multilingual language model," https://www.sarvam.ai/blogs/sarvam-m, 23 May 2025, 2025.

[5] ——, "Open-sourcing Sarvam 30B & 105B models," https://www.sarvam.ai/blogs/sarvam-30b-105b, 6 March 2026, 2026.

[6] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, K. Erk and N. A. Smith, Eds. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 1715–1725. [Online]. Available: https://aclanthology.org/P16-1162/

[7] T. Kudo, "Subword regularization: Improving neural network translation models with multiple subword candidates," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, I. Gurevych and Y. Miyao, Eds. Melbourne, Australia: Association for Computational Linguistics, Jul. 2018, pp. 66–75. [Online]. Available: https://aclanthology.org/P18-1007/

[8] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, E. Blanco and W. Lu, Eds. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 66–71. [Online]. Available: https://aclanthology.org/D18-2012/

[9] OpenAI, "tiktoken: a fast BPE tokeniser for use with OpenAI's models," https://github.com/openai/tiktoken, 2023, encodings `cl100k_base` and `o200k_base`.

[10] A. Grattafiori, A. Dubey, A. Jauhri *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[11] P. Rust, J. Pfeiffer, I. Vulić, S. Ruder, and I. Gurevych, "How good is your tokenizer? on the monolingual performance of multilingual language models," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, C. Zong, F. Xia, W. Li, and R. Navigli, Eds. Online: Association for Computational Linguistics, Aug. 2021, pp. 3118–3135. [Online]. Available: https://aclanthology.org/2021.acl-long.243/

[12] V. Zouhar, C. Meister, J. Gastaldi, L. Du, M. Sachan, and R. Cotterell, "Tokenization and the noiseless channel," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 5184–5207. [Online]. Available: https://aclanthology.org/2023.acl-long.284/

[13] D. Kakwani, A. Kunchukuttan, S. Golla, G. N.C., A. Bhattacharyya, M. M. Khapra, and P. Kumar, "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 4948–4961. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.445/

[14] S. Doddapaneni, R. Aralikatte, G. Ramesh, S. Goyal, M. M. Khapra, A. Kunchukuttan, and P. Kumar, "Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 12 402–12 426. [Online]. Available: https://aclanthology.org/2023.acl-long.693/

[15] S. Khanuja, D. Bansal, S. Mehtani *et al.*, "MuRIL: Multilingual representations for Indian languages," arXiv:2103.10730, 2021.

[16] J. Gala, P. A. Chitale, R. AK *et al.*, "IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages," *Transactions on Machine Learning Research*, 2023. [Online]. Available: https://openreview.net/forum?id=vfT4YuzAYA

[17] M. S. U. R. Khan, P. Mehta, A. Sankar, U. Kumaravelan, S. Doddapaneni, S. B, V. G, S. Jain, A. Kunchukuttan, P. Kumar, R. Dabre, and M. M. Khapra, "IndicLLMSuite: A blueprint for creating pre-training and fine-tuning datasets for Indian languages," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 15 831–15 879. [Online]. Available: https://aclanthology.org/2024.acl-long.843/

[18] N. Goyal, C. Gao, V. Chaudhary, P.-J. Chen, G. Wenzek, D. Ju, S. Krishnan, M. Ranzato, F. Guzmán, and A. Fan, "The Flores-101 evaluation benchmark for low-resource and multilingual machine translation," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 522–538, 2022. [Online]. Available: https://aclanthology.org/2022.tacl-1.30/

[19] NLLB Team, M. R. Costa-jussà, J. Cross *et al.*, "No language left behind: Scaling human-centered machine translation," arXiv:2207.04672, 2022.

[20] M. Brahma, N. J. Karthika, R. Verma *et al.*, "Multilingual tokenization through the lens of Indian languages: Challenges and insights," arXiv:2506.17789; accepted to Findings of ACL 2026, 2025.

[21] M. Brahma, N. J. Karthika, A. Singh *et al.*, "MorphTok: Morphologically grounded tokenization for Indian languages," arXiv:2504.10335; Tokenization Workshop (TokShop) at ICML 2025, 2025.

[22] S. Rana, A. Menezes, A. Kulkarni *et al.*, "MUTANT: A recipe for multilingual tokenizer design," arXiv:2511.03237 (v1 titled "IndicSuperTokenizer: An Optimized Tokenizer for Indic Multilingual LLMs"), 2025.

[23] The Unicode Consortium, "Unicode Standard Annex #29: Unicode text segmentation," https://www.unicode.org/reports/tr29/, 2025.

[24] Mistral AI, "Mistral NeMo (introducing the Tekken tokenizer)," https://mistral.ai/news/mistral-nemo, 18 July 2024, 2024.

[25] Gemma Team, A. Kamath, J. Ferret *et al.*, "Gemma 3 technical report," arXiv:2503.19786, 2025.